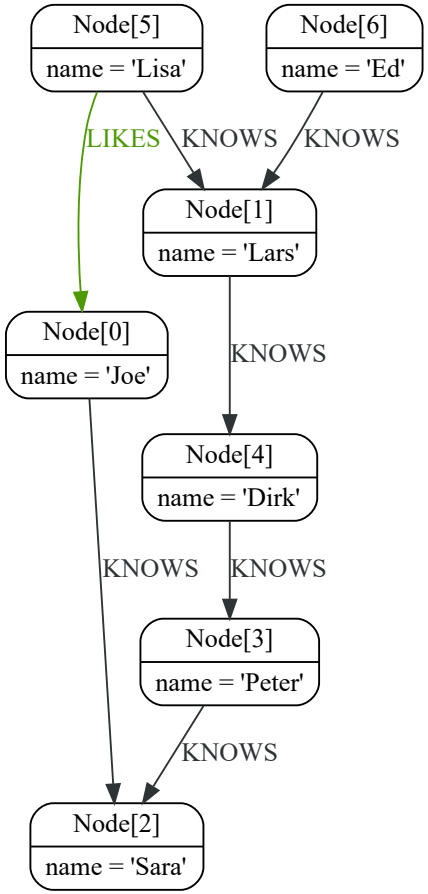
Traversal Framework examples

The following are some examples of how you can use the Traversal Framework. The source code for the examples can be found in `TraversalExample.java` ↗.



This graph illustrates a small group of friends with the definition of `RelationshipType`:

```
private enum Rels implements RelationshipType
{
    LIKES, KNOWS
}
```

Traversing a graph examples

The graph can be traversed with, for example, the following traverser starting at the node with the `name = 'Joe'`:

```
for ( Path position : db.traversalDescription()
        .depthFirst()
        .relationships( Rels.KNOWS )
        .relationships( Rels.LIKES, Direction.INCOMING )
        .evaluator( Evaluators.toDepth( 5 ) )
        .traverse( node ) )
{
    output += position + "\n";
}
```

The traversal will thus output:

```
(0)
(0)<-[LIKES,1]-(5)
(0)<-[LIKES,1]-(5)-[KNOWS,6]->(1)
(0)<-[LIKES,1]-(5)-[KNOWS,6]->(1)<-[KNOWS,5]-(6)
(0)<-[LIKES,1]-(5)-[KNOWS,6]->(1)-[KNOWS,4]->(4)
(0)<-[LIKES,1]-(5)-[KNOWS,6]->(1)-[KNOWS,4]->(4)-[KNOWS,3]->(3)
(0)<-[LIKES,1]-(5)-[KNOWS,6]->(1)-[KNOWS,4]->(4)-[KNOWS,3]->(3)-[KNOWS,2]->(2)
```

Since `TraversalDescription` is immutable, it is useful to create template descriptions that include common settings shared by different traversals. For example, start with this traverser:

```
friendsTraversal = db.traversalDescription()
        .depthFirst()
        .relationships( Rels.KNOWS )
        .uniqueness( Uniqueness.RELATIONSHIP_GLOBAL );
```

It yields the following output (starting from the node with the `name = 'Joe'`):

```
(0)
(0)-[KNOWS,0]->(2)
(0)-[KNOWS,0]->(2)<-[KNOWS,2]-(3)
(0)-[KNOWS,0]->(2)<-[KNOWS,2]-(3)<-[KNOWS,3]-(4)
(0)-[KNOWS,0]->(2)<-[KNOWS,2]-(3)<-[KNOWS,3]-(4)<-[KNOWS,4]-(1)
```

```
(0)-[KNOWS,0]->(2)<-[KNOWS,2]-(3)<-[KNOWS,3]-(4)<-[KNOWS,4]-(1)<-[KNOWS,6]-(5)
(0)-[KNOWS,0]->(2)<-[KNOWS,2]-(3)<-[KNOWS,3]-(4)<-[KNOWS,4]-(1)<-[KNOWS,5]-(6)
```

Then, create a new traverser from it, restricting depth to three:

```
for ( Path path : friendsTraversal
        .evaluator( Evaluators.toDepth( 3 ) )
        .traverse( node ) )
{
    output += path + "\n";
}
```

This should be the output:

```
(0)
(0)-[KNOWS,0]->(2)
(0)-[KNOWS,0]->(2)<-[KNOWS,2]-(3)
(0)-[KNOWS,0]->(2)<-[KNOWS,2]-(3)<-[KNOWS,3]-(4)
```

In case you want to traverse from depth 2 to 4:

```
for ( Path path : friendsTraversal
        .evaluator( Evaluators.fromDepth( 2 ) )
        .evaluator( Evaluators.toDepth( 4 ) )
        .traverse( node ) )
{
    output += path + "\n";
}
```

This gives the following output:

```
(0)-[KNOWS,0]->(2)<-[KNOWS,2]-(3)
(0)-[KNOWS,0]->(2)<-[KNOWS,2]-(3)<-[KNOWS,3]-(4)
(0)-[KNOWS,0]->(2)<-[KNOWS,2]-(3)<-[KNOWS,3]-(4)<-[KNOWS,4]-(1)
```

For other useful evaluators, see Using Evaluators.

The `Traverser` also has a `nodes()` method, which returns an `Iterable` of all the nodes in the paths:

```
for ( Node currentNode : friendsTraversal
        .traverse( node )
        .nodes() )
{
    output += currentNode.getProperty( "name" ) + "\n";
}
```

This gives the following output:

```
Joe
Sara
Peter
Dirk
Lars
Lisa
Ed
```

You can do this with relationships as well. Here is an example:

```
for ( Relationship relationship : friendsTraversal
        .traverse( node )
        .relationships() )
{
    output += relationship.getType().name() + "\n";
}
```

```
KNOWS
KNOWS
KNOWS
KNOWS
KNOWS
KNOWS
```

Implementing a user-defined procedure

This example shows how to implement a user-defined procedure using the Traversal Framework. The transaction and logger are made available through the Procedure Framework:

```
@Context
public Transaction tx;
```

```
@Context
public Log log;

@Procedure(value = "traverse.findPeople")
@Description("Finds all the known people to the given Person")
public Stream<PathResult> findFriends(@Name("person") Node person) {

    final Traverser traverse = tx.traversalDescription()
            .breadthFirst()
            .relationships(Relatic                                  "), Direction.OUTGOING)
            .evaluator(Evaluators.
                                                View all (13 more lines)
            .evaluator(new PathLogger())
```

This allows the Traversal Framework to be used side by side with Cypher:

```
MATCH (p:Person { name: 'Joe' })
CALL traverse.findPeople(p) YIELD path RETURN [friend IN nodes(path) | friend.name] AS
friends
```

Prev
‹ Bidirectional Traversal Framework

Next
Transaction management ›

Contents

Traversing a graph examples
Implementing a user-defined procedure

LEARN

Sandbox
Neo4j Community Site
Neo4j Developer Blog
Neo4j Videos

SOCIAL

Twitter
Meetups
Github
Stack Overflow

CONTACT US →

US: 1-855-636-4532
Sweden +46 171 480 113
UK: +44 20 3868 3223
France: +33 (0) 1 88 46 13 20

GraphAcademy
Want to Speak?
Neo4j Labs

© 2026 Neo4j, Inc.
Terms | Privacy | Sitemap

Neo4j®, Neo Technology®, Cypher®, Neo4j® Bloom™ and Neo4j® Aura™ are registered trademarks of Neo4j, Inc. All other marks are owned by their respective companies.